

Yahya Izadi

2025-11-24

:
Theory) () (Approximation
.

“ ”
.
(n) ($n-1$)
(Runge's Phenomenon) .

: Overfitting

Overfitting
(Test Data) .

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange

# ( )
x = np.array([1, 2, 3, 4, 5])
y = np.array([2.1, 3.9, 8.2, 15.8, 24.5])

# (Overfitting) -
```

```

poly_lagrange = lagrange(x, y)
x_new = np.linspace(0.5, 5.5, 100)
y_lagrange = poly_lagrange(x_new)

# ( )
coefs = np.polyfit(x, y, 2)
y_polyfit = np.polyval(coefs, x_new)

plt.figure(figsize=(10, 6))
plt.scatter(x, y, color='red', s=100, label='Data Points', zorder=5)
plt.plot(x_new, y_lagrange, 'g--', linewidth=2, label='Lagrange Interpolation (Overfitting)')
plt.plot(x_new, y_polyfit, 'b-', linewidth=3, label='Quadratic Approximation (Generalizable)')

plt.title("Runge Phenomenon vs. Good Approximation")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

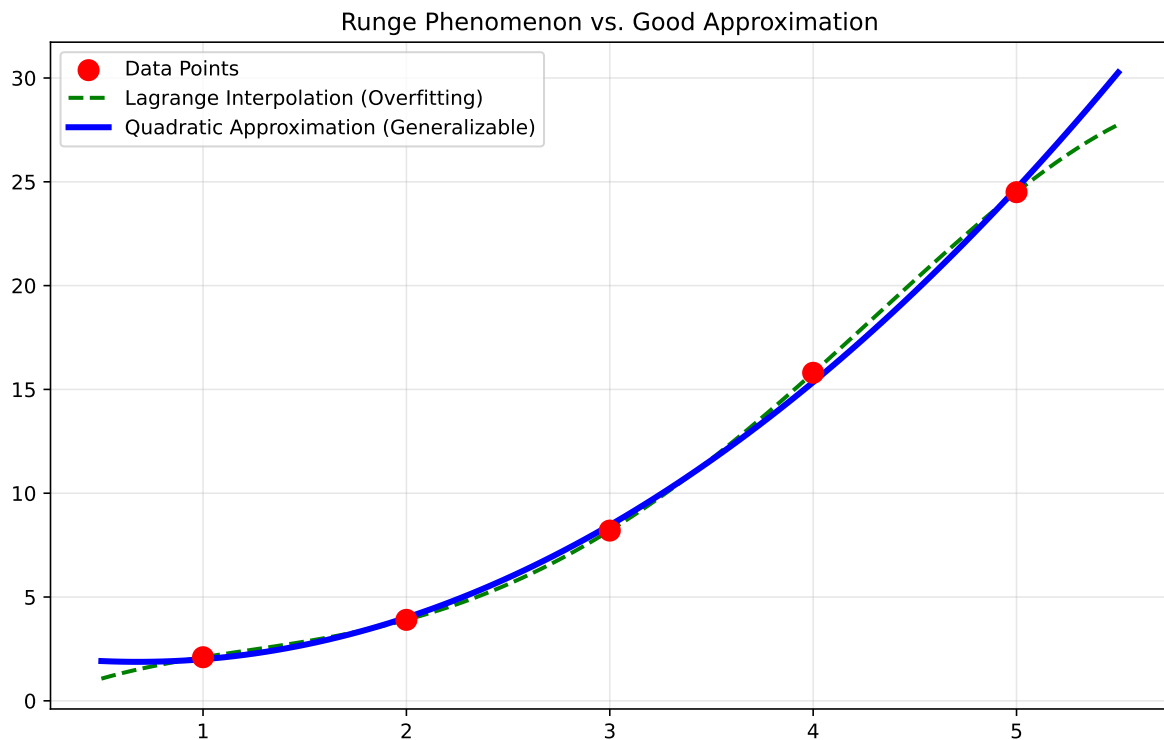


Figure 1: (Overfitting) (Good Fit)

(Cubic Splines): “ ”

().

: **(Smoothing)**

(ECG) .

- **GAMs (Generalized Additive Models):**

```
from scipy.interpolate import UnivariateSpline

# ( )
x_noise = np.linspace(0, 10, 30)
y_true = np.sin(x_noise)
y_noise = y_true + np.random.normal(0, 0.2, size=len(x_noise))

# (Smoothing Spline)
spl = UnivariateSpline(x_noise, y_noise)
spl.set_smoothing_factor(0.5)

x_smooth = np.linspace(0, 10, 100)

plt.figure(figsize=(10, 5))
plt.plot(x_noise, y_noise, 'ro', alpha=0.5, label='Noisy Data')
plt.plot(x_smooth, spl(x_smooth), 'k-', linewidth=3, label='Cubic Spline Smoothing')
plt.plot(x_noise, y_true, 'b:', label='True Function (Hidden)')
plt.legend()
plt.title("Data Smoothing using Cubic Splines")
plt.grid(True, alpha=0.3)
plt.show()
```

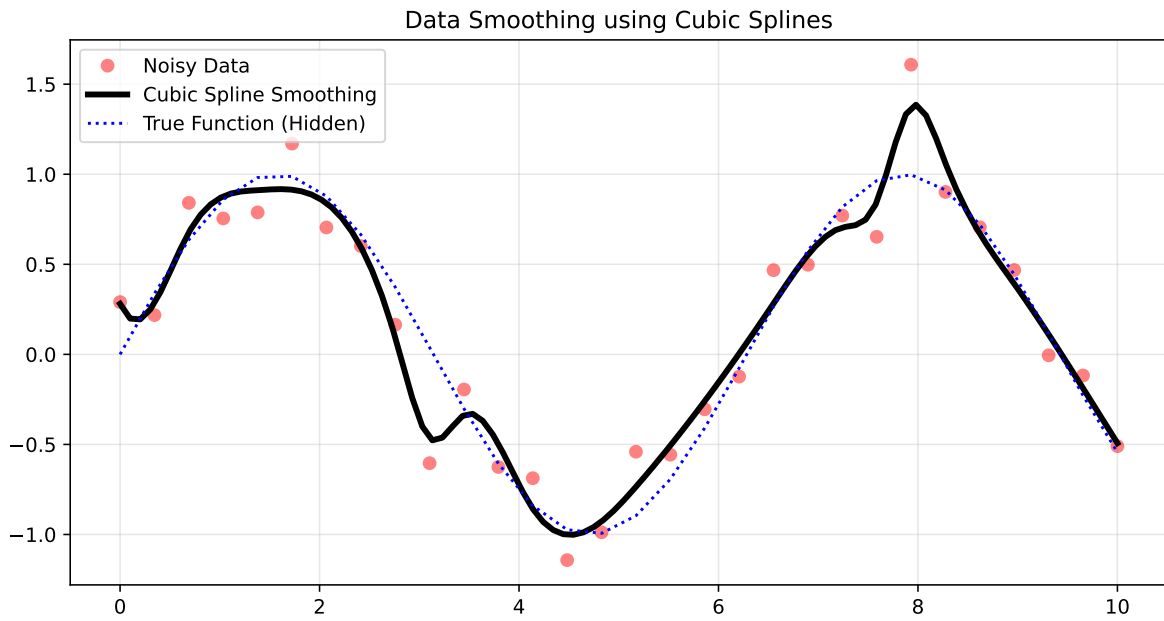


Figure 2:

. :
 .
 () .
 : **Active Learning**
 (()) : “
 () .”

. **(Finite Differences)**
 :

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

: Gradient Checking

PyTorch TensorFlow (Deep Learning) (Gradient) .
 . Gradient Checking . Backpropagation

	Overfitting ()
(Spline)	Data Smoothing / GAMs
	Active Learning ()
	Gradient Checking
